

**Kannan**

Posted on Sep 24, 2020



Filter an array for unique values in Javascript

#javascript #node #codenewbie #webdev

Here are a few ways to filter unique values in javascript.

1. Using Set:

Set is a new data structure introduced in ES6. It is somewhat similar to an array but it does not allow us to store duplicate values.

```
const duplicates = ['🐼', '🐱', '🐹', '🐻', '🐼', '🐹'];  
const unique = Array.from(new Set(duplicates));  
console.log(unique);  
/* ['🐼', '🐱', '🐹', '🐻'] */
```

To filter unique values we pass an array to the `Set` constructor and we frame back an array from the Set object using `Array.from` method.

Set works perfectly with the string and number types. But it will not work with objects why? 🤔

Take a look at this example:

```
const duplicates = [{name: 'Tommy', value: '🐼'},
                    {name: 'Max', value: '🐱'},
                    {name: 'Lucy', value: '🐭'},
                    {name: 'Cooper', value: '🐻'},
                    {name: 'Tommy', value: '🐼'},
                    {name: 'Lucy', value: '🐭'}];

const unique = Array.from(new Set(duplicates));

console.log(unique);
/*
[
  {name: 'Tommy', value: '🐼'},
  {name: 'Max', value: '🐱'},
  {name: 'Lucy', value: '🐭'},
  {name: 'Cooper', value: '🐻'},
  {name: 'Tommy', value: '🐼'},
  {name: 'Lucy', value: '🐭'},
]
*/
```

Even though Tommy and Lucy are duplicated. Set was unable to identify them as duplicates.

Take a look at another example:

```
const tommy = {name: 'Tommy', value: '🐶'};
const max = {name: 'Max', value: '🐱'};
const lucy = {name: 'Lucy', value: '🐱'};
const cooper = {name: 'Cooper', value: '🐱'};

const uniqueSet = new Set();

uniqueSet.add(tommy);
uniqueSet.add(max);
uniqueSet.add(lucy);
uniqueSet.add(cooper);
uniqueSet.add(tommy);
uniqueSet.add(lucy);

console.log(uniqueSet);
/*
[
  {name: 'Tommy', value: '🐶'},
  {name: 'Max', value: '🐱'},
  {name: 'Lucy', value: '🐱'},
  {name: 'Cooper', value: '🐱'}
]
*/

uniqueSet.add({name: 'Tommy', value: '🐶'});

console.log(uniqueSet);
/*
[
  {name: 'Tommy', value: '🐶'},
  {name: 'Max', value: '🐱'},
  {name: 'Lucy', value: '🐱'},
  {name: 'Cooper', value: '🐱'},
  {name: 'Tommy', value: '🐶'}
]
*/
```

This is happening because when objects are compared they are compared by reference and not by value. That's why when lucy and tommy are added at first they were not added because they share the same reference. But when I added the tommy value directly it would have created a new reference.

2. Using Filter/Reduce:

```
const duplicates = [
  {name: 'Tommy', value: '🐼'},
  {name: 'Max', value: '🐱'},
  {name: 'Lucy', value: '🐼'},
  {name: 'Cooper', value: '🐱'},
  {name: 'Tommy', value: '🐼'},
  {name: 'Lucy', value: '🐼'},
];

const unique = duplicates.filter((elem, index) =>
  duplicates.findIndex(obj => obj.value === elem.value) === index);
```

We can also use `Array.reduce` to achieve the same kind of result.

Here the idea is `findIndex` or `indexOf` method would return the index of the first element that matches the criteria. So if the index of `filter` is not equal to the index returned by `findIndex` then the element is a duplicate.

This type will work for all types but for every element in array, we have to loop through the array again. Imagine where you want to filter in thousands of records. This approach is not an efficient one $O(n^2)$.

3. Flag Approach:

```
const duplicates = [
  {name: 'Tommy', value: '🐼'},
  {name: 'Max', value: '🐱'},
  {name: 'Lucy', value: '🐶'},
  {name: 'Cooper', value: '🐱'},
  {name: 'Tommy', value: '🐼'},
  {name: 'Lucy', value: '🐶'},
];

const removeDuplicates = (duplicates) => {
  const flag = {};
  const unique = [];
  duplicates.forEach(elem => {
    if (!flag[elem.value]) {
      flag[elem.value] = true;
      unique.push(elem);
    }
  });
  console.log(flag);
  // { '🐼': true, '🐱': true, '🐶': true, '🐱': true }
  return unique;
}

const unique = removeDuplicates(duplicates);
```

This is my favorite way to find unique especially when dealing with a large array of objects. We will loop through the array and in the flag object we will check if the value is already present. If not we will add the value to the flag object and push it to the array else we will just ignore it. In this, we will loop through the array only once so time complexity will be $O(n)$.

What is your favorite way and why? Let us know in the comments.

Top comments (3) ⌵

 Viznij Roman • Oct 16 '22

✕

```
function uniq(arr) {  
  let result = {}  
  arr.map(el=>{  
    result[el.name] = el  
  })  
  return Object.values(result)  
}
```

Code of Conduct • Report abuse

Trending in Node

The Node.js community is currently discussing the new features in Node.js 20.6.0, especially the built-in support for `.env` files, making the 'dotenv' package less necessary. Other topics of interest include Node.js architectural patterns, the unveiling of Bun 1.0, a new tool that combines a runtime environment, package manager, and bundler, and the use of Node.js with OpenAI and ModelFusion to create chatbots. There's also a focus on building multi-tenant architecture in Node.js.

#node #webdev #javascript #react